Studying Cross Transferability of Vision Transformers using HAM10000 skin cancer dataset

by Azwad Tamir

Azwad Tamir
Published in Machine Intelligence and Deep Learning
15 min read · May 1, 2022

Github link: *https://github.com/azwad-tamir/Transfer_ViT*

YouTube Video link: https://www.youtube.com/watch?v=ivfOPi2GmwQ

Introduction: The number of learnable parameters of deep learning models have risen substantially in recent years. Some very large deep learning models can contain more than 10^13 parameters which require a long time, many GPUs, and a large dataset containing many datapoints to train effectively. However, there are many practical situations where such resources are not readily available. For example, in bioinformatics, datapoints have to be labeled using domain experts making large datasets scarce and often not made open source. One solution to this problem is through Transfer learning which involves training a model twice on two different datasets. The process is made up of two steps. First is the pretraining step which involves training the model on a large opensource dataset. The next step is finetuning, where the pretrained model is finetuned on the smaller application dataset. Finetuning could be done in two ways: 1) All the model parameters are updated during the finetuning process; 2) The initial layers of the model are frozen and only the layers towards the end are updated. The latter process is more widely accepted and would be used in this study.

One of the limitations of transfer learning is that the two datasets involved in the transfer learning process should be similar in nature. Otherwise, problems such as negative learning and overfitting may occur. However, getting a model which is pretrained on a dataset similar to the application dataset is often not an option.

The objective of this study is to determine if there are significant advantages to applying transfer learning when the two datasets involved are different in nature. Also, we would investigate whether it is better to train the model in a conventional way directly on the application dataset without transfer learning if pretrained models similar to the application dataset could not be found.

Dataset: The two datasets used in this study are the ImageNet and the Ham10000. ImageNet consists of images of everyday objects like cars, animals, furniture, etc. It has around 14 million datapoints and 10000 classes. This was used for pretraining in this study. Examples of ImageNet images are given in Fig 1. On the other hand, the Ham10000 dataset is made up of images of skin lesions where the label is the type of cancer associated with the skin lesion. The dataset has 10,015 samples, where each image has a resolution of 600x450. There are seven classes and the ground truth has been verified by domain experts. This dataset would be used to finetune the models in this study. The dataset is split into training, validation, and test sets where the training set consists of 85% of the data, while the validation and test set each have 7.5% of the data. Examples of images from the Ham10000 dataset are given in Fig 2.

The two datasets used for pretraining and finetuning and purposely selected to be very different in nature and has very little in common so that we could investigate how much, if any, benefit we can get from transfer learning for vision transformers if the pretraining and finetuning datasets are not similar at all.

Fig.1 Examples of ImageNet classes [2]

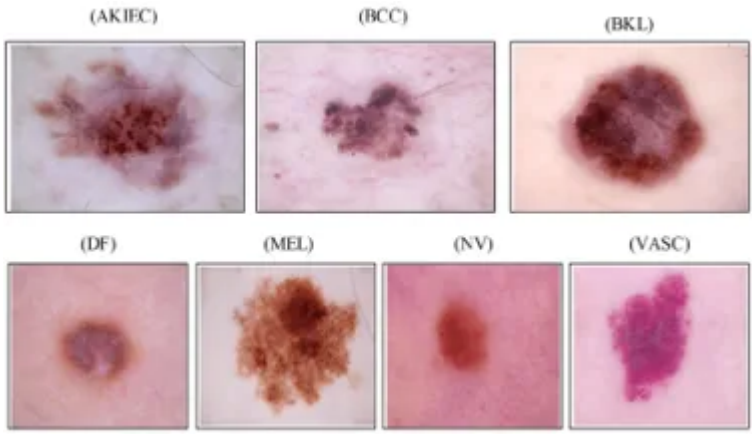


Fig.2 Examples of skin lesions from Ham10000 [1]

Benchmark Models: To understand the difficulty in training the Ham10000 dataset and to establish baselines, four different CNN-based vision classifier models have been trained on the Ham10000 dataset. The benchmark models chosen are VGGNet, ResNet, DenseNet, and InceptionV3.

VGG_Net: The basic architecture of the VGG_Net is shown in Fig 3. It consists of a combination of convolutional layers with RELU activation and max pooling layers. There are a number of fully connected layers at the end with a softmax layer to make the classification head. There are 5 convolutional layer blocks with batch normalization. The PyTorch programming framework was used to implement the model and the pretrained model was imported from TorchVision. The input data had a resolution of 224x224. The data transformations incorporated in preprocessing were random crop, random horizontal flip, and normalization. The cross-entropy loss

function was used with Stochastic Gradient Descent (SGD) optimizer, a batch size of 16 was chosen and the learning rate was 0.001. The model was trained for 27 epochs. The hyperparameters for this model and all other models trained in this study are figured out with the help of a grid search algorithm and the values obtaining the best accuracy on the validation set are chosen.

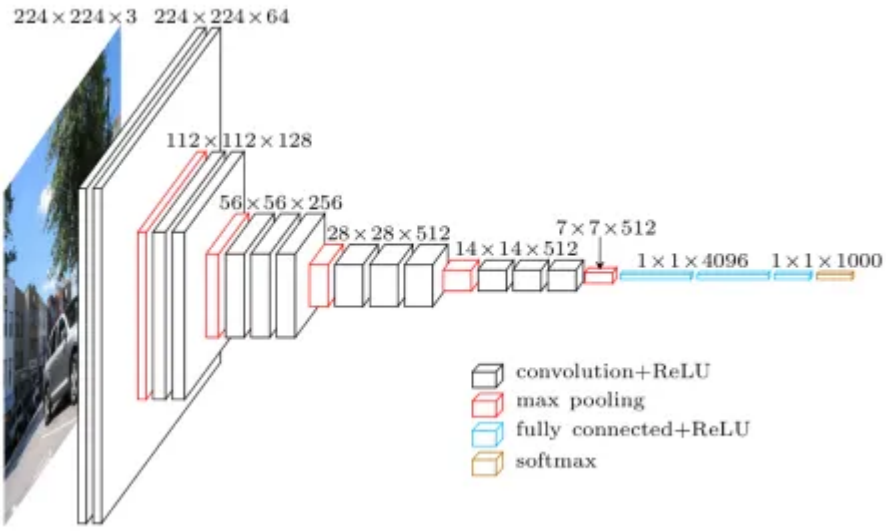


Fig.3 Basic architecture of the VGGNet model [3]

ResNet: The next benchmark model trained was ResNet152. It consists of 152 layers in total and the basic architecture is shown in Fig 4. It is a modification of the VGGNet with residual connections bypassing each convolutional block. This skip connections help the model to train better and solve the problem of vanishing and exploding gradients which is often seen in very deep models. The PyTorch programming framework was used to implement the model and the pretrained model was imported from TorchVision. The data transformations incorporated in preprocessing were random crop, random horizontal flip, and normalization. The input images were 224x224. The cross-entropy loss function was used with Stochastic Gradient Descent (SGD) optimizer, a batch size of 32 was chosen and the learning rate was 0.001. The model was trained for 25 epochs.

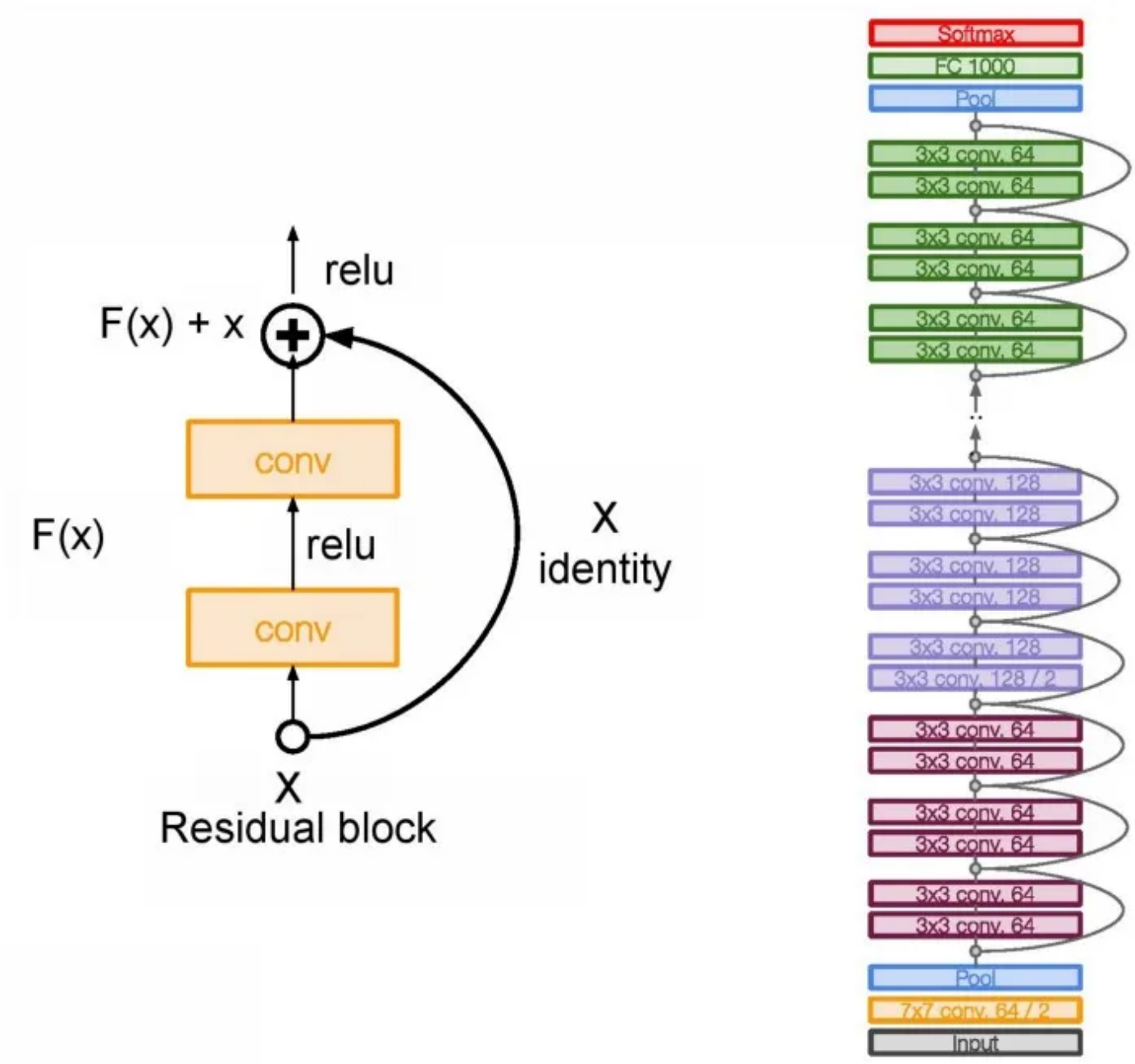


Fig. 4 Basic architecture of the ResNet model [4]

DenseNet: The next model trained was DenseNet. It is a modification of the ResNet with a dense network of residual connections going from each layer to all layers in front of it. This made the model more efficient, lowered complexity, get more diversified features, and a strong gradient flow. The model has 161 layers in total. The PyTorch programming framework was used to implement the model and the pretrained model was imported from TorchVision. The data transformations incorporated in preprocessing were random crop, random horizontal flip, and normalization. The input images had a resolution of 224x224. The cross-entropy loss

function was used with Stochastic Gradient Descent (SGD) optimizer, a batch size of 32 was chosen and the learning rate was 0.001. The model was trained for 17 epochs.

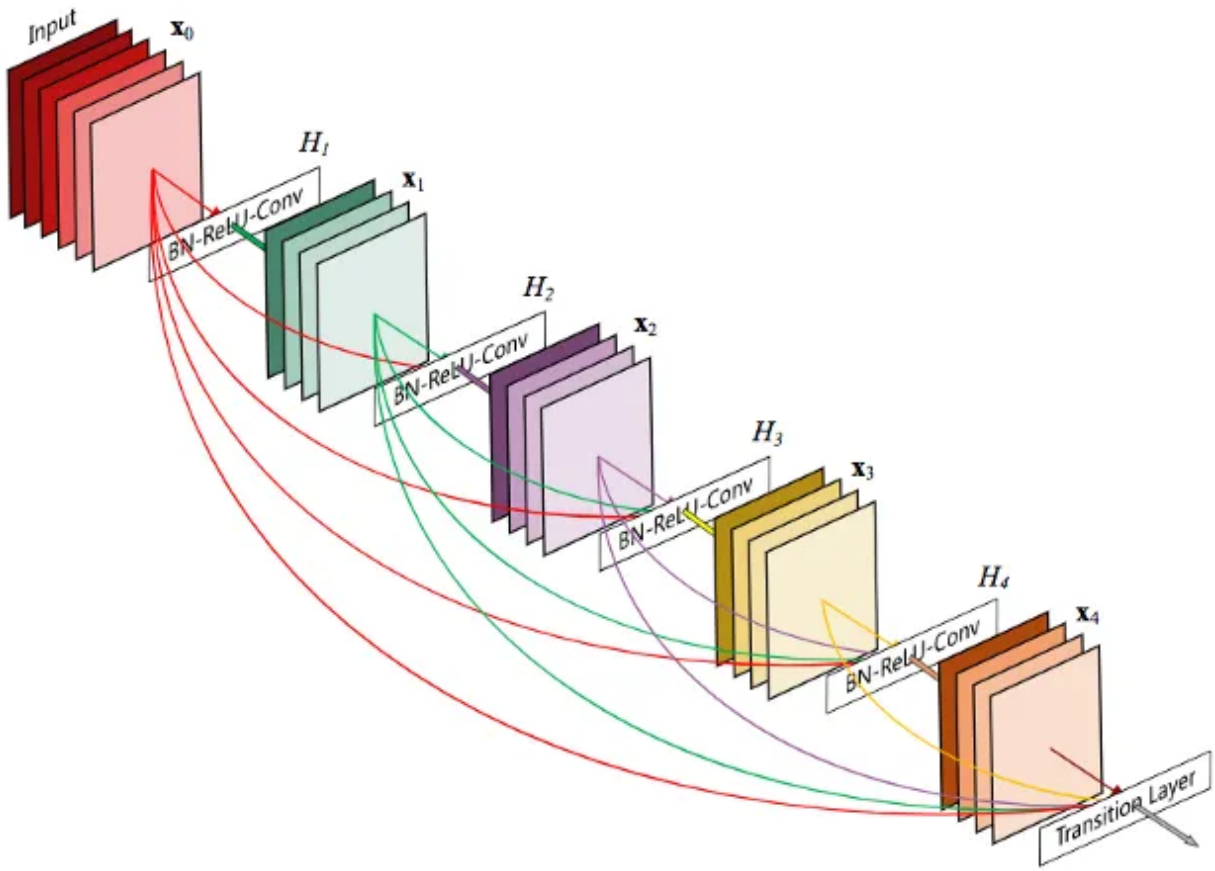


Fig. 5 Basic architecture of the DenseNet model [6]

InceptionV3: The last benchmark model was inceptionV3. This model used parallel connection to learn more diversified representation which helped it to gain more accuracy. The model also used auxiliary classifiers making it more efficient and lower computational complexity. The PyTorch programming framework was used to implement the model and the pretrained model was imported from TorchVision. The data transformations incorporated in preprocessing were random crop, random horizontal flip, and normalization. The input images had a resolution of 299x299. The cross-entropy loss function was used with Stochastic Gradient Descent (SGD) optimizer, a batch size of 32 was chosen and the learning rate was 0.001. The model was trained for 23 epochs.

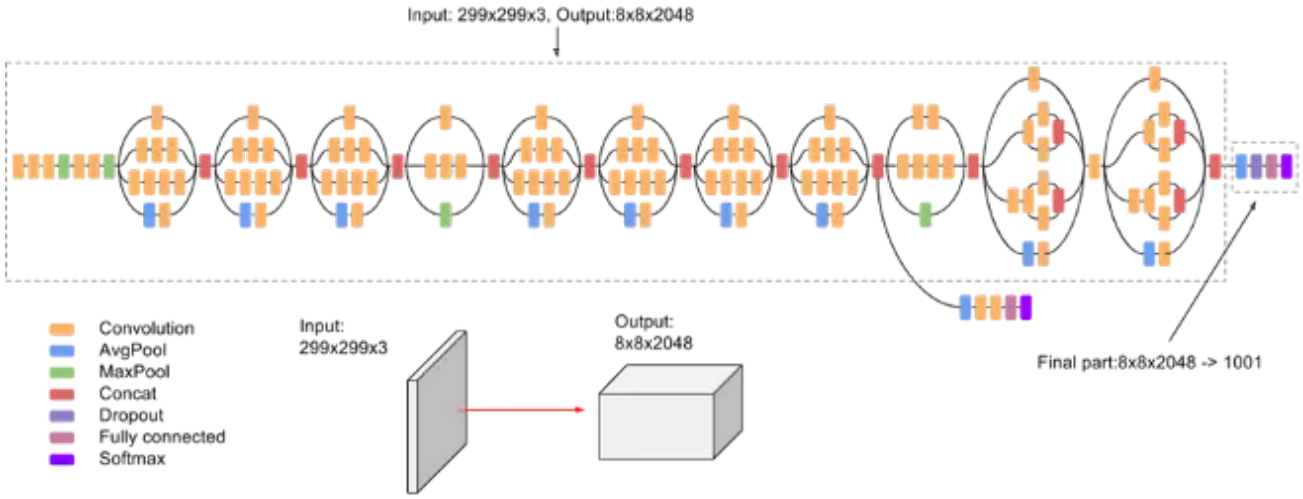


Fig. 6 Basic architecture of the InceptionV3 model [6]

Transformers: This section gives an overview of the transformer model and the basic architecture of the Vision Transformer. The transformer model was first introduced in the paper “Attention is all you need”[7]. It was a sequence to sequence model that was primarily made for performing NLP tasks. It consists of an encoder and a decoder which are made up of multi head attention layers, normalization layers and feed forward layers. The multi head self attention layers are what made it unique compared to the other previous NLP deeplearning algorithms. Later on, different research groups modified the transformer architecture to fit various applications. These modified models could be divided into three major groups. First, there are the encoder only models which only consist of an encoder. An example of this is the BERT model which could be used for classification and regression tasks. The second type are the decoder only models which include architectures like GPT2. These could be used for text generation and lastly, we have the encoder-decoder models like the T5 which could be used to do tasks like machine translations.

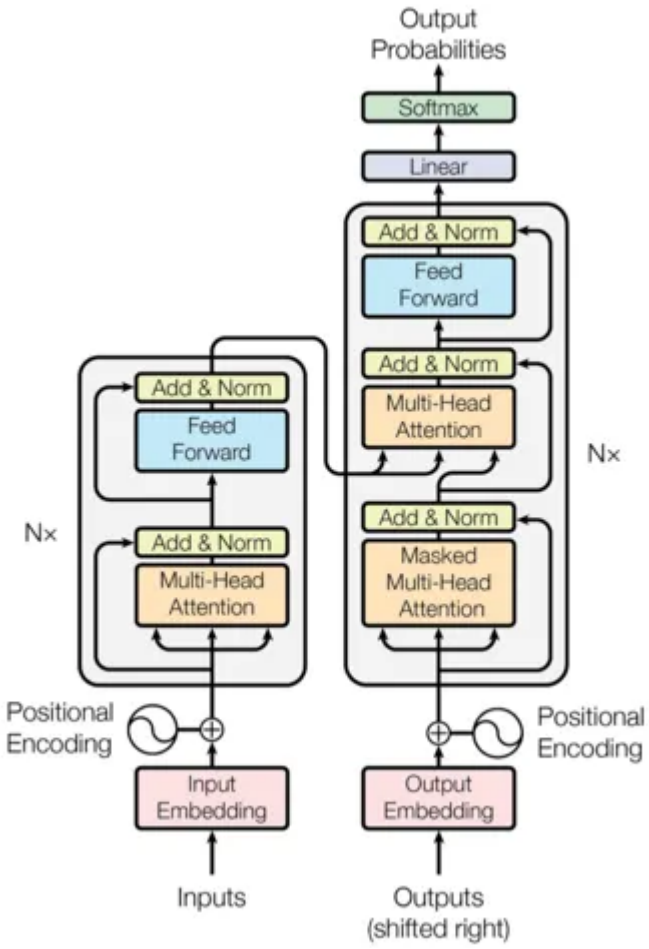


Fig. 7 The Transformer architecture [7]

Vision Transformer (ViT): The vision transformer is a Bert like model so it only has an encoder. The working principle of ViT is similar to Bert but it takes on images instead of sequences. The tokenization method involves splitting the image into patches and then flattening them. Next, positional encoding is added to the patches and these tokens are then pushed into the transformer encoder, which consists of a normalization layer followed by the multihead attention layer, the multilayer perceptron, and a classification head at the end.

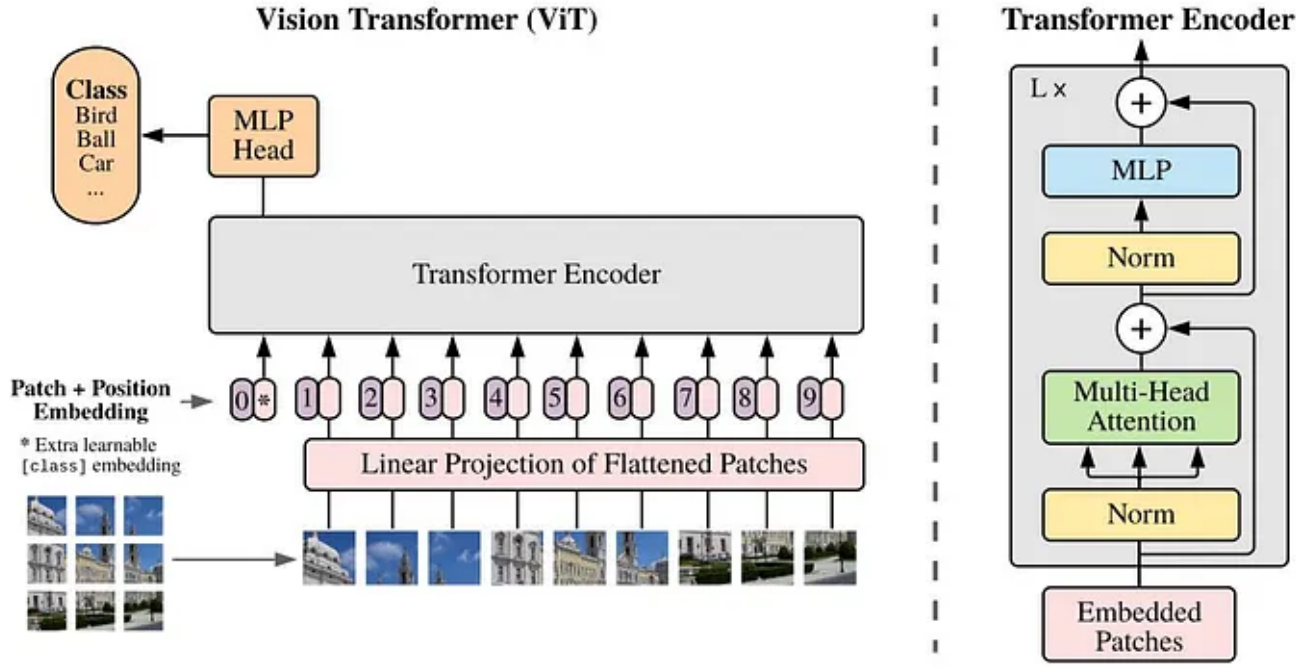


Fig. 8 The basic architecture of the vision Transformer (ViT) [8]

Base Models: The next part of the project involves training four different vision transformer models from scratch without transfer learning on the Ham10000 dataset. The results of these would be later compared to models with transfer learning to understand the benefits of transfer learning on vision transformers where the pretraining and finetuning datasets are not similar to each other. The vision transformer models trained in this way are the Vit model which is the original vision transformer, DeepVit, T2TViT and CaiT. The model architectures of the latter three models are discussed below:

DeepViT: This model replaced the self attention layer of the ViT with a re-attention layer which helps to train deeper models effectively [9]. Vision transformers often face the problem of attention collapse. This happens in deeper models when the information flow gets disrupted and the attention parameters fail to learn. This is similar to the vanishing and exploding gradient problem seen in other deep learning models. DeepVit alleviated this problem by recomputing the attention maps to increase diversity. It does this with the help of a theta matrix that gets multiplied with the attention values in each layer. The formula used to calculate the re-attention is given in Fig 11.

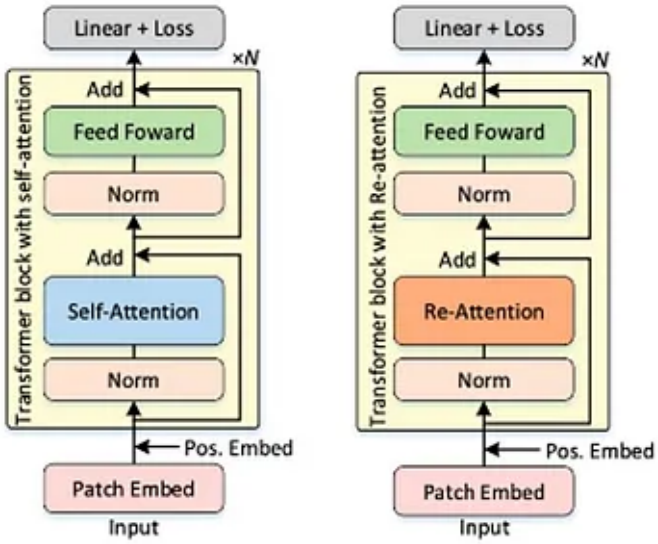


Fig. 9 Vit vs DeepVit model architecture

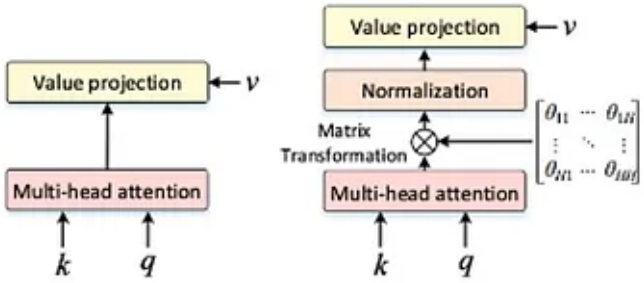


Fig. 10 Multihead attention layers of Vit and DeepVit model

$$\text{Re}-\text{Attention}(Q,K,V)=\text{Norm}\left(\Theta^{\top}\left(\text{Softmax}\left(\frac{QK^{\top}}{\sqrt{d}}\right)\right)\right)V$$

Fig. 11 Re-attention layer activation equation [9]

T2TViT: This model incorporates an alteration in the tokenization method to increase the accuracy. The algorithm recursively aggregates neighboring tokens into single ones which progressively learns the representation instead of learning it all at once. In successive steps, it reorganizes the tokens and folds them into grids. It later applies a square window to select neighboring tokens and flattens them into different combinations of the image patches. This reduces the number of tokens and makes it lighter and easier to train. It also diversifies the representation learned making the entire algorithm more efficient increasing accuracy. The entire token to token process is demonstrated in Fig 12.

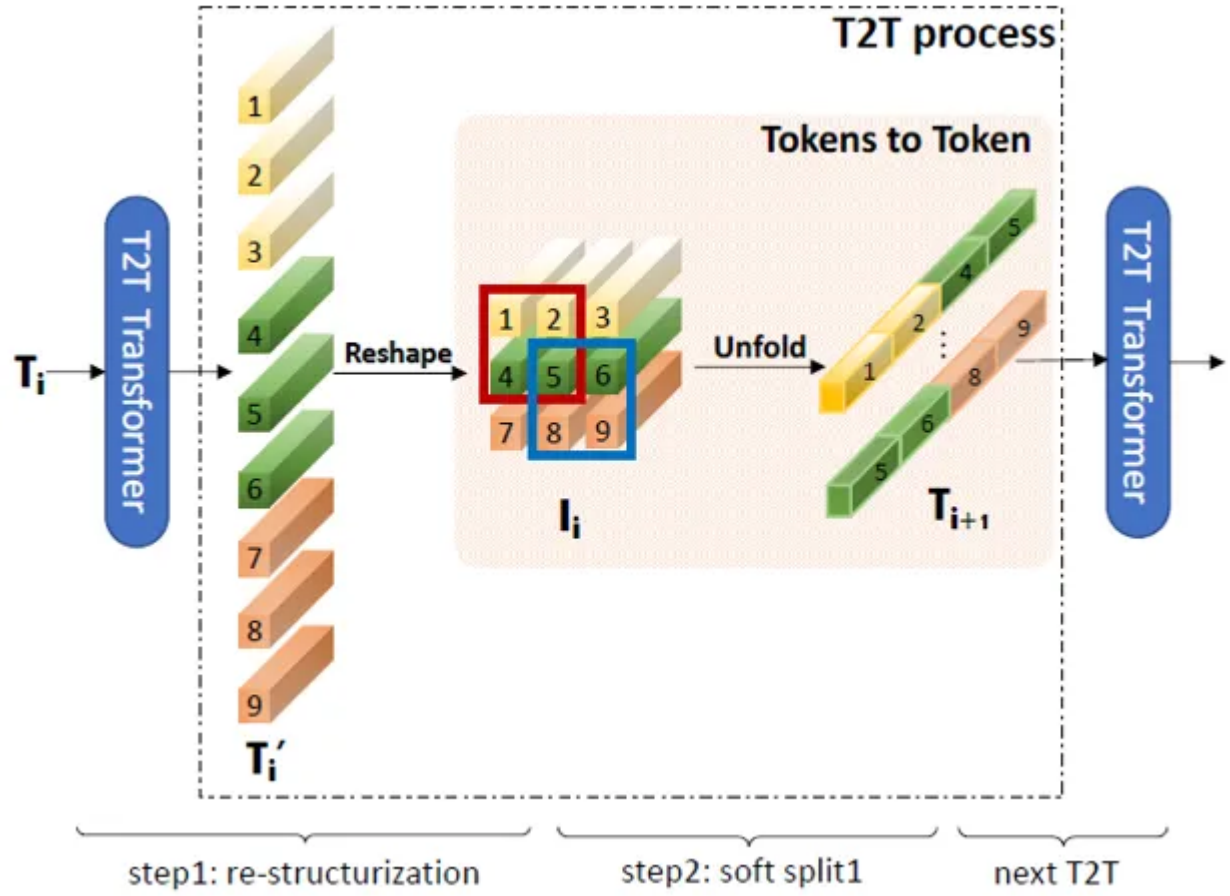


Fig. 12 Novel tokenization process of the T2T transformer model [10]

CaiT: This model includes a separate activation pathway called class attention layers to process the class embeddings. A vision transformer needs to process two types of information simultaneously. One is learning the self attention between the patches. The other one uses the image features and the linear classifiers to produce the class embeddings. It is often difficult for the model to optimize these two objectives simultaneously, especially during the training process. The CaiT model tries to alleviate this problem by separating these two learning targets. First, it learns the self attention layers potioned in the early parts of the model. After that, it freezes the self attention layers and inputs the class tokens into the later layers after the self attention blocks. This fixes the patch embedding and the model devotes its entirety in training the class embeddings. This modification to the architecture makes this model more efficient and shows better accuracy compared to the original ViT model on the ImageNet dataset. Fig 13 shows the distinction in the architecture of the ViT model compared to the CaiT model.

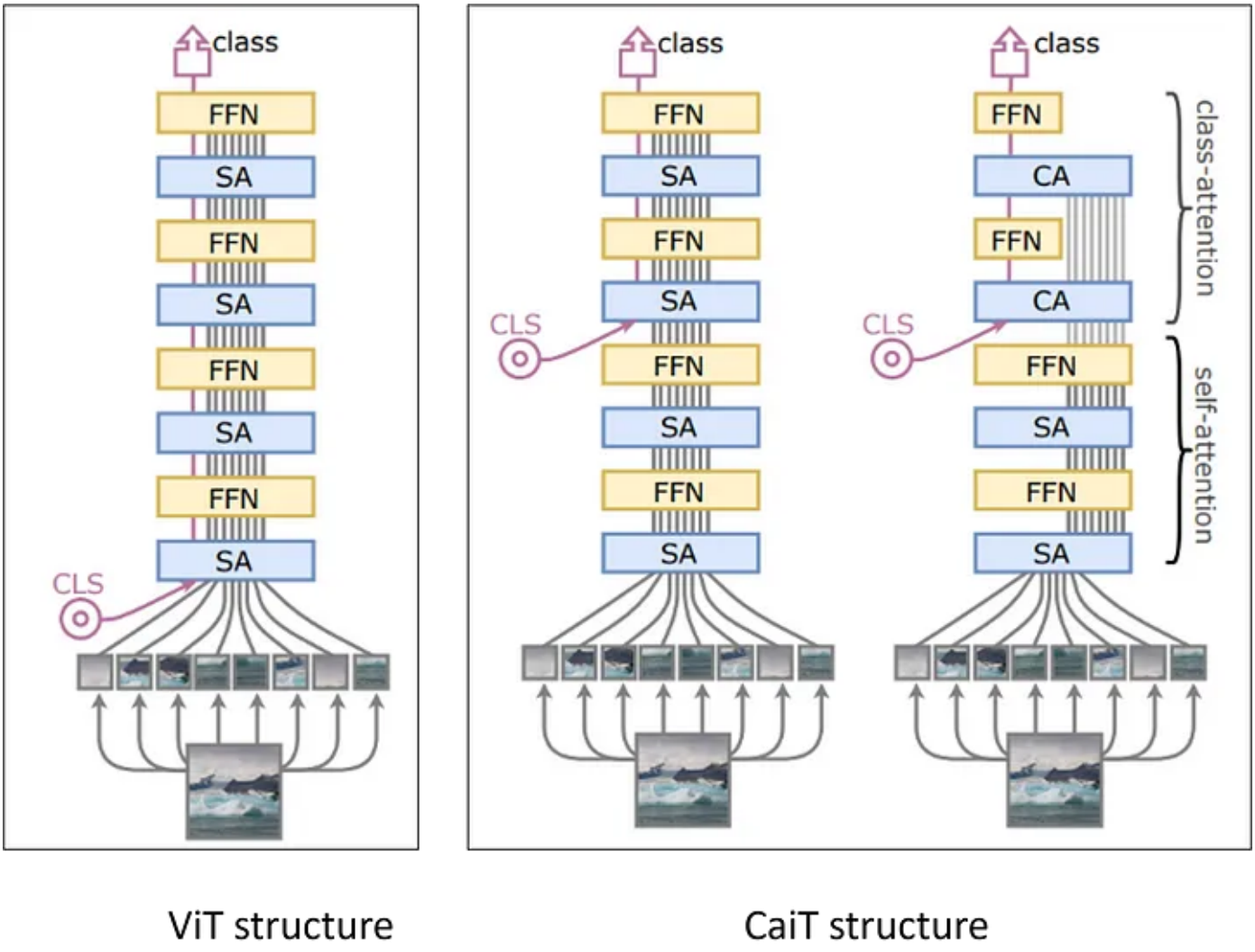


Fig. 13 Comparison of the CaiT structure with the original ViT model [11]

Vision Transformers Fine Tuned: In the next part of the project, three different vision transformer models are trained using transfer learning. As mentioned above, the ImageNet dataset is used to pretrain the models followed by a finetune process with the Ham10000 dataset. The pretrained models are downloaded from the HuggingFace website. We also used to HuggingFace training API to finetune the vision transformers. The three models that were trained using transfer learning are the original ViT model, the DeiT model, and the BeiT model. The basic architecture of the DeiT and BeiT models are explained below.

DeiT: This modification of the vision transformer was proposed in the paper "Training data-efficient image transformers & distillation through attention" paper [12]. The advantage of this model is that it requires less data in order to train compared to the original transformer model. It consists of two separate networks, the teacher and the

student network. A distillation token is included at the end of the patch embeddings. The real network is trained to predict the output of the teacher network instead of the true labels. This makes the training process more efficient compared to the original ViT model. Two different teacher networks are tried out in the paper. The CNN based ResNet model worked better as the teacher network compared to using another vision transformer in the experiment. As a result we have implemented ResNet50 as the teacher network.

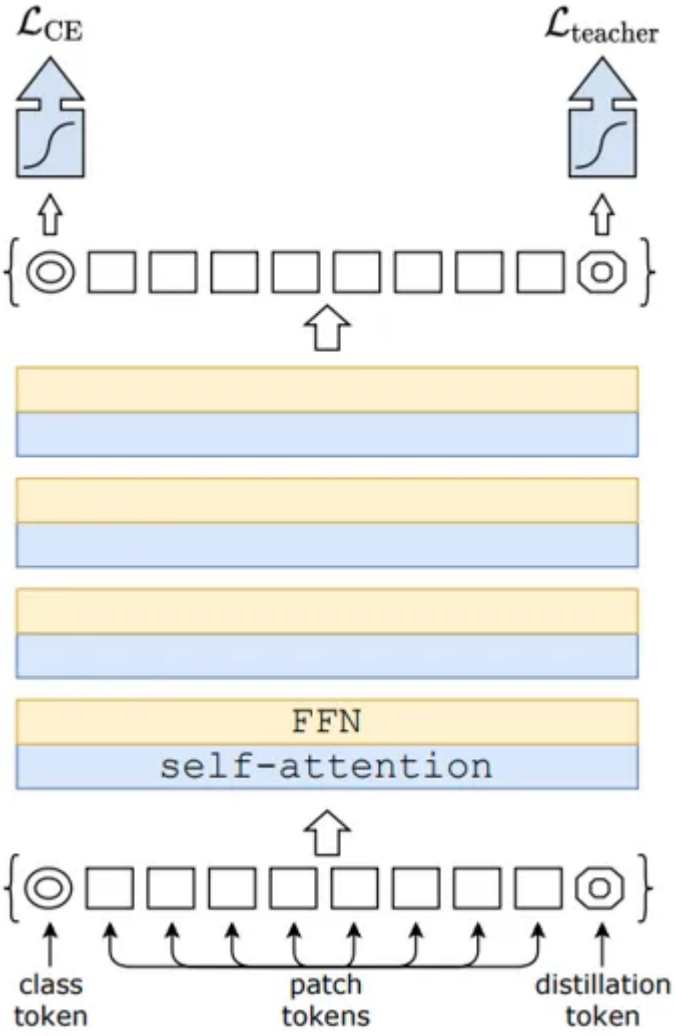


Fig. 14 High level representation of the DeiT model [12]

BeiT: This is the last pretrained model that was implemented. This uses a Bert type self-supervised training during the pretraining process instead of using supervised learning. In this case, after the image patches are flattened, several random patches are hidden with the help of a mask. The masked patches along with the normal ones are then fed to the transformer encoder and the target of the transformer is to predict these masked image patches. The number of patches that are masked is a

hyperparameter and needs to be tuned to find the best results. The advantage of this kind of pretraninig is that unlabeled images could also be used to train the model which comes in handy when a large labeled dataset is not available. The authors of this study showed that the self-supervised process is just as effective as the supervised form of training used in the case of the original vision transformer.

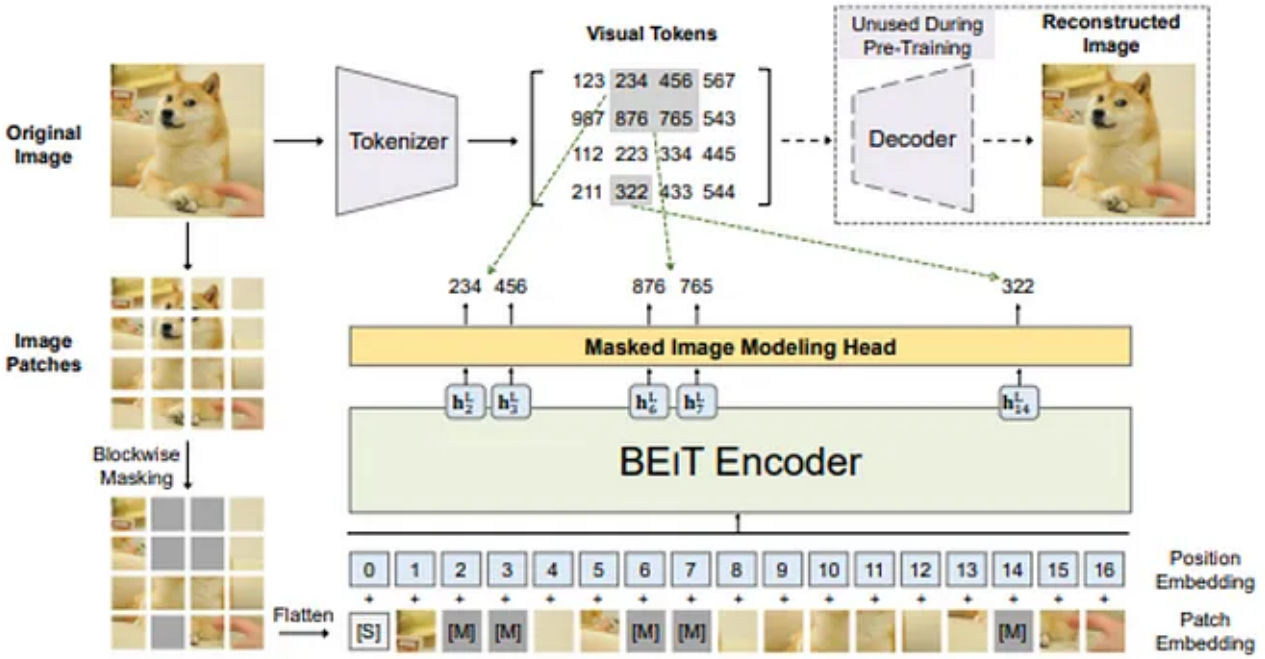


Fig. 15 Basic architecture of the Beit Model [13]

Results: This part reports the results that were obtained in the study. Fig 16 shows the accuracy, precision, and F1 scores of the benchmark models. ResNet obtained the best accuracy of 91.01% with a F1 score of 0.90. The other benchmark models also showed similar performance with VGGNet obtaining the worst accuracy of 88.92%. Fig 17 shows the confusion matrix for the benchmark models. The diagonal elements in the confusion matrix represent the correctly predicted samples while the non-diagonal elements show the False positives and True negatives. Some of the classes in the Ham10000 dataset contain very little data while others consist of the vast majority. This further makes the dataset harder to train and lowers the accuracy.

Fig. 16 Accuracy metrics of benchmark models. (a)VGGNet, (b)ResNet (c)DenseNet (d)InceptionV3

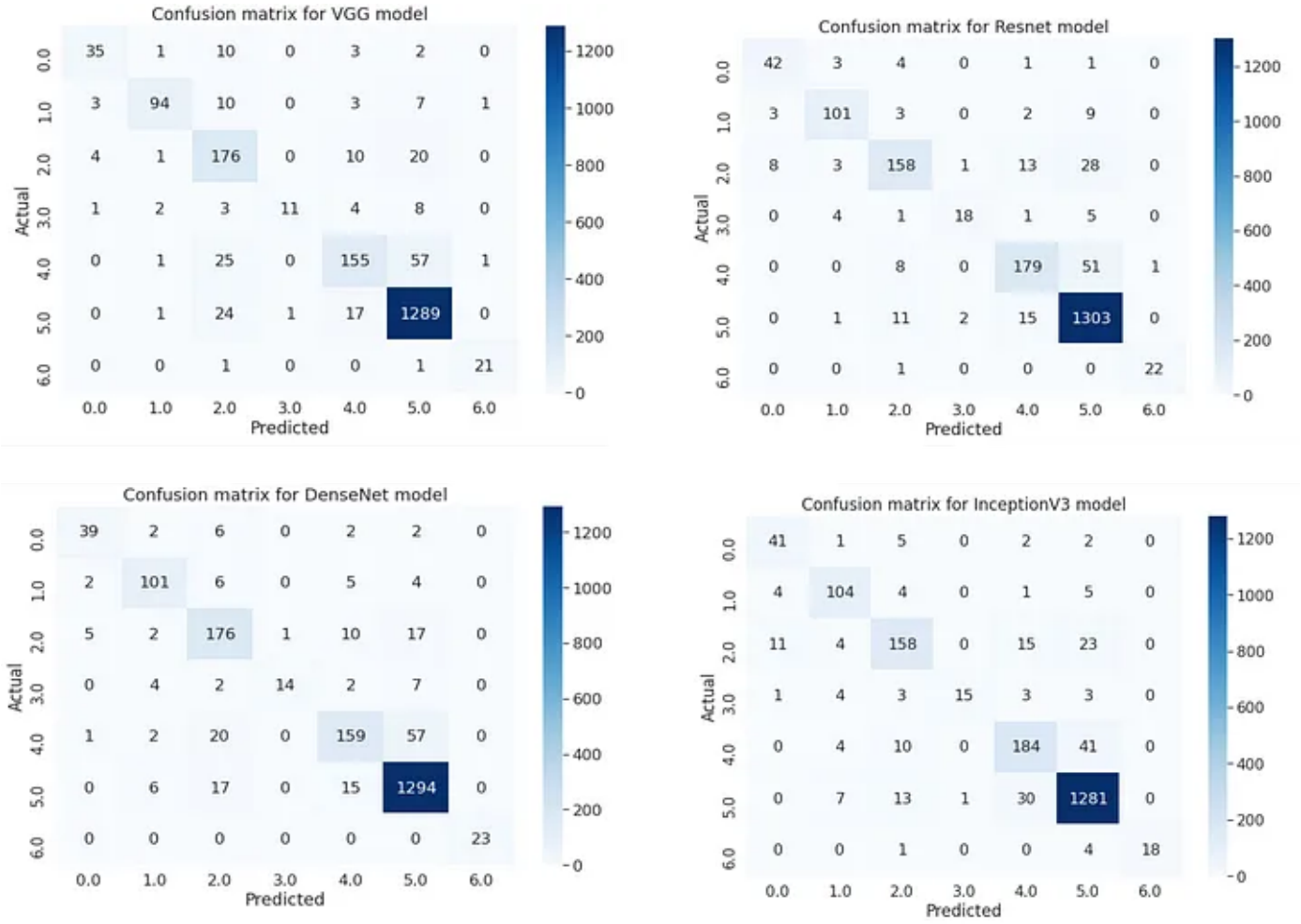


Fig. 17 Confusion matrix for the benchmark models

Next, the accuracies of the base models are shown in Fig 18 with the confusion matrix given in Fig 19. These are the models that were trained without any transfer learning. The accuracy together with the precision, recall and F1 score are significantly lower compared to the benchmark results. This is due to the fact that the vision transformer models consist of a large number of trainable parameters and the Ham10000 dataset is not large enough to effectively train the models properly. This is even more true for the classes that have less numbers of samples. The accuracy for the ViT model is 72% with deepViT, CaiT and T2TViT achieving 73.40%, 73.80% and 75.27% accuracies respectively. The precision, recall, and F1 scores of these models are also significantly lower than the benchmark models. Looking at the confusion matrix, it is quite evident that the classes with very few samples did not train at all and most of the data belonging to these classes were wrongly predicted as other classes.

Fig. 18 Accuracy metrics of transformer models without transfer learning. (a)ViT, (b)DeepViT, ©CaiT, (d)T2TViT

Fig. 19 Confusion matrix for the base models

Lastly, the performance of the pretrained models on the test set is given in Fig 20. The results show a significant boost in accuracy for the pretrained models compared to the base models. The original ViT model got an accuracy of 87.77% while the DeiT and the BeiT model got 86.70% and 83.75% accuracies respectively. The F1 scores for pretranined models are also significantly greater than the base models and could be said to be comparable to the benchmark models. The confusion matrix for the pretrained models are given in Fig 21. The results show much better accuracies for the low data classes as well with the majority of the predictions lying on the diagonal.

Fig. 20 Accuracy metrics of transformer models with transfer learning. (a)ViT, (b)DeiT, (c)BeiT

Fig. 21 Confusion matrix for Pretrained models

Training: The training curve for the base models and the pretrained models are given in Fig 22 and Fig 23 respectively. The graph has number of steps on the x-axis and accuracy on the y-axis. The number of steps is directly proportional to the number of epochs where each epoch is equal to the training sample size divided by the batch size. Each graph shows a steady increase in the training accuracy for all the base models but the validation accuracy saturates very early on. So the models are overfitted and confirm the previous observation that the number of samples in the dataset is not enough to train the models. However, the training curve for the pretrained models shows better performance with the validation accuracy saturating off at a much higher accuracy compared to the base models.

Fig. 22 Training curve for the base models

Fig. 23 Training curve for the pretrained models

Inference: Several inferences could be drawn from the experimental results. Firstly, it is seen that transfer learning helps to improve the results of vision transformers significantly even if the datasets are not similar to each other. This goes against the common conjecture in deep learning literature that the datasets must be similar to avoid negative learning and get good results for transfer learning. The most likely cause of this is due to the fact that transformers are generalized models that not only learn the dataset but also learn the task that needs to be accomplished in a particular situation. So here, the vision transformer first learns that it is a classification task and then figures out how to differentiate the various classes. So, although the datasets are not similar, understanding that the objective of the model is not dependent on the contents of the images, so transfer learning helps to teach the model about that fact. However, such a conclusion that vision transformers behave differently to transfer

learning compared to other deep learning models needs further study and experiments in order to be comprehensively proven. This opens up opportunity for further work in this domain.

Other inferences that could be drawn from the data is that, modifications to the vision transformer architecture do not provide much advantage when the training dataset size is low as the different models trained in this study showed similar performance on the test dataset. Moreover, even with transfer learning, the vision transformer models still showed worse performance compared to the CNN based benchmark models. This exposes the limitation of vision transformer models when the application dataset size is low. Hence, future work is necessary to build more robust transformer models that could handle low data applications.

Conclusion: Transformers are very large deeplearning models that require a large amount of data to train effectively. This study explores the possibility of applying transfer learning to vision transformers even when the pretraining and finetuning datasets are not similar. The results show significant performance benefits when transfer learning is applied to vision transformers even when the pretraining dataset and finetuning datasets are very different. However, the results show that even with transfer learning, vision transformers still lack in performance compared to CNN-based models when the application dataset size is low. This opens up opportunity for further research in transformer architecture to make it more robust and efficient and able to train with a limited amount of training data. *

Reference:

[1] M. Khan, M. Sharif, T. Akram, R. Damasevicius, and R. Maskeliunas, “Skin Lesion Segmentation and Multiclass Classification Using Deep Learning Features and Improved Moth Flame Optimization,” *Diagnostics,* vol. 11, p. 811, 04/29 2021, doi: 10.3390/diagnostics11050811.

[2] T. Ye, “Visual Object Detection from Lifelogs using Visual Non-lifelog Data,” 2018.

[3] https://www.pyimagesearch.com/2017/03/20/imagenet-vggnet-resnet-inception-xception-keras/

[4] http://cs231n.stanford.edu/slides/2019/cs231n_2019_lecture09.pdf

[5] https://medium.com/@sh.tsang/review-inception-v3-1st-runner-up-image-classification-in-ilsvrc-2015-17915421f77c

[6] https://towardsdatascience.com/architecture-comparison-of-alexnet-vggnet-resnet-inception-densenet-beb8b116866d

[7] A. Vaswani *et al.*, "Attention is All you Need," presented at the Advances in Neural Information Processing Systems, 2017, 2017. [Online]. Available: https://proceedings.neurips.cc/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf.

[8] A. Dosovitskiy *et al.*, "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale," 2020 2020, doi: 10.48550/ARXIV.2010.11929.

[9] D. Zhou *et al.*, "DeepViT: Towards Deeper Vision Transformer," 2021 2021, doi: 10.48550/ARXIV.2103.11886.

[10] L. Yuan *et al.*, "Tokens-to-Token ViT: Training Vision Transformers from Scratch on ImageNet," 2021 2021, doi: 10.48550/ARXIV.2101.11986.

[11] H. Touvron, M. Cord, A. Sablayrolles, G. Synnaeve, and H. Jégou, "Going deeper with Image Transformers," 2021 2021, doi: 10.48550/ARXIV.2103.17239.

[12] H. Touvron, M. Cord, M. Douze, F. Massa, A. Sablayrolles, and H. Jégou, "Training data-efficient image transformers & distillation through attention," 2020 2020, doi: 10.48550/ARXIV.2012.12877.

[13] H. Bao, L. Dong, and F. Wei, "BEiT: BERT Pre-Training of Image Transformers," 2021 2021, doi: 10.48550/ARXIV.2106.08254.

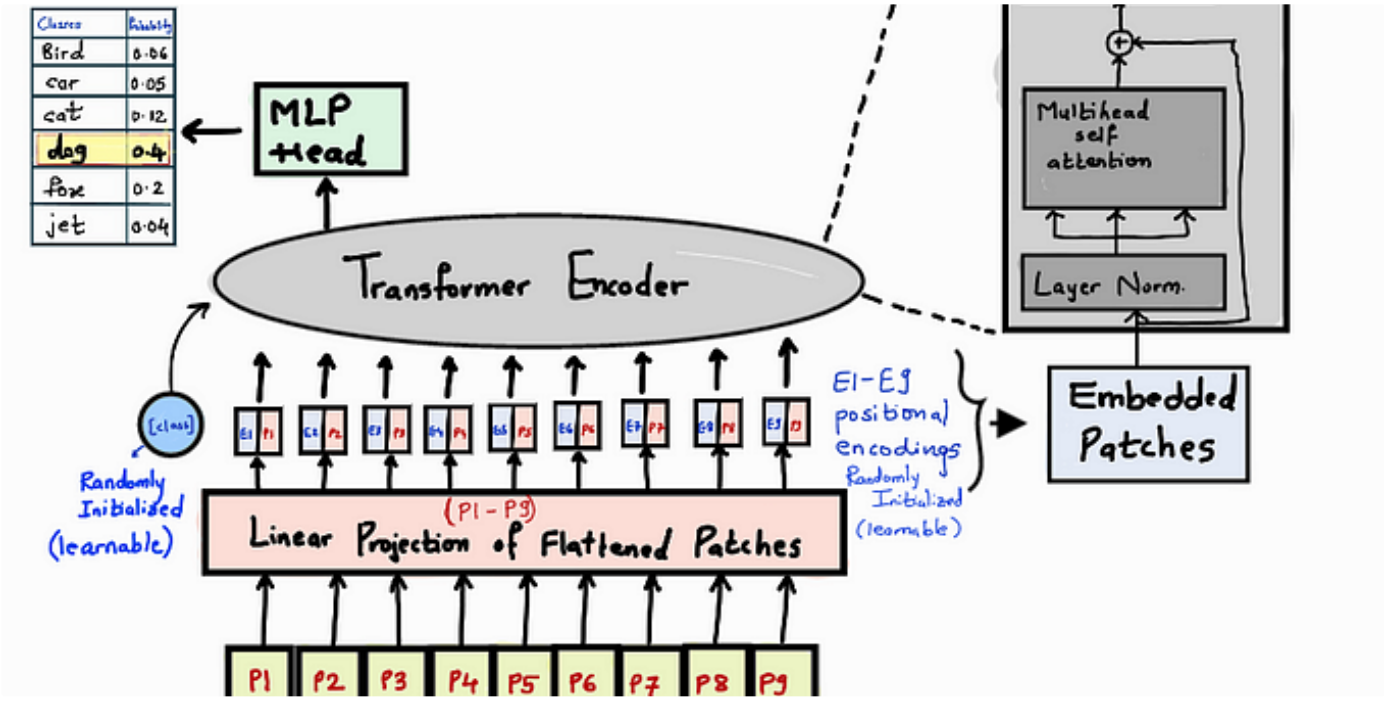
Vision Transformer | Transfer Learning | Imagenet

Edit profile

Written by Azwad Tamir

1 Follower · Writer for Machine Intelligence and Deep Learning

More from Azwad Tamir and Machine Intelligence and Deep Learning



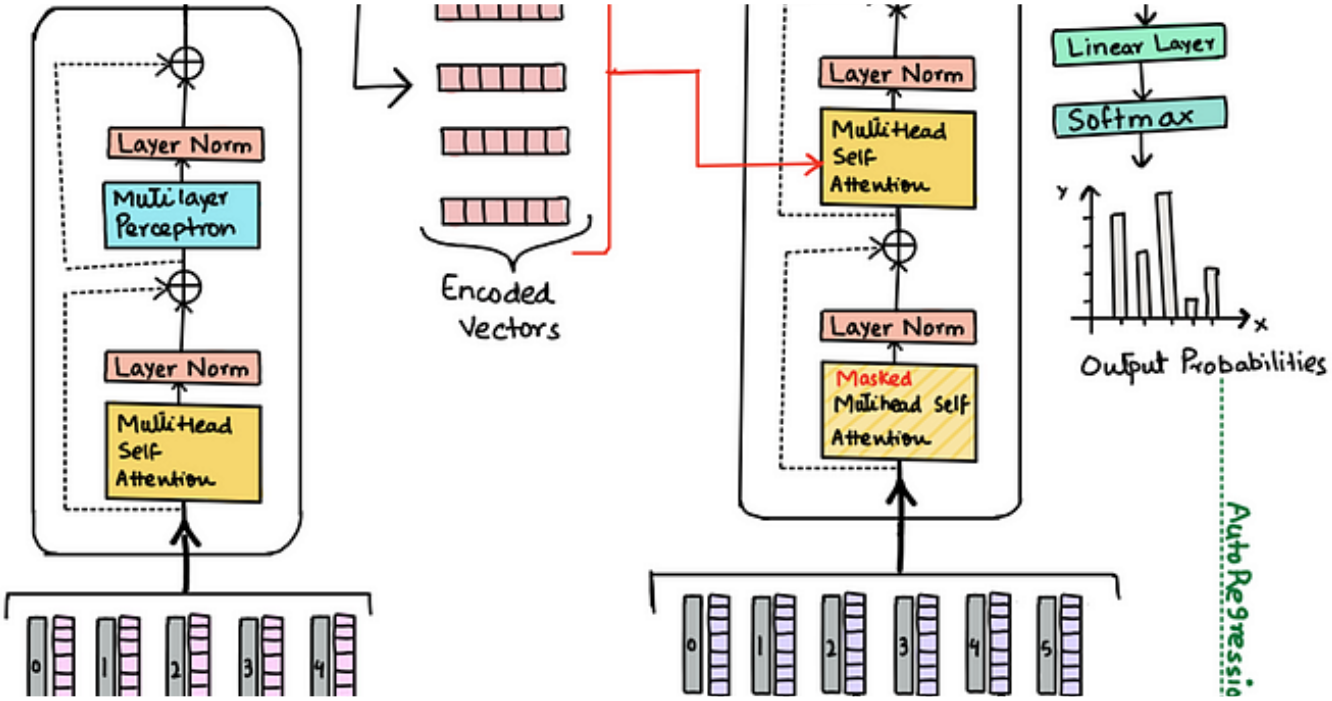
Shivani Junawane in Machine Intelligence and Deep Learning

ViT: Vision Transformer

Transformers for image recognition at scale.

9 min read · Apr 23, 2022

205 3



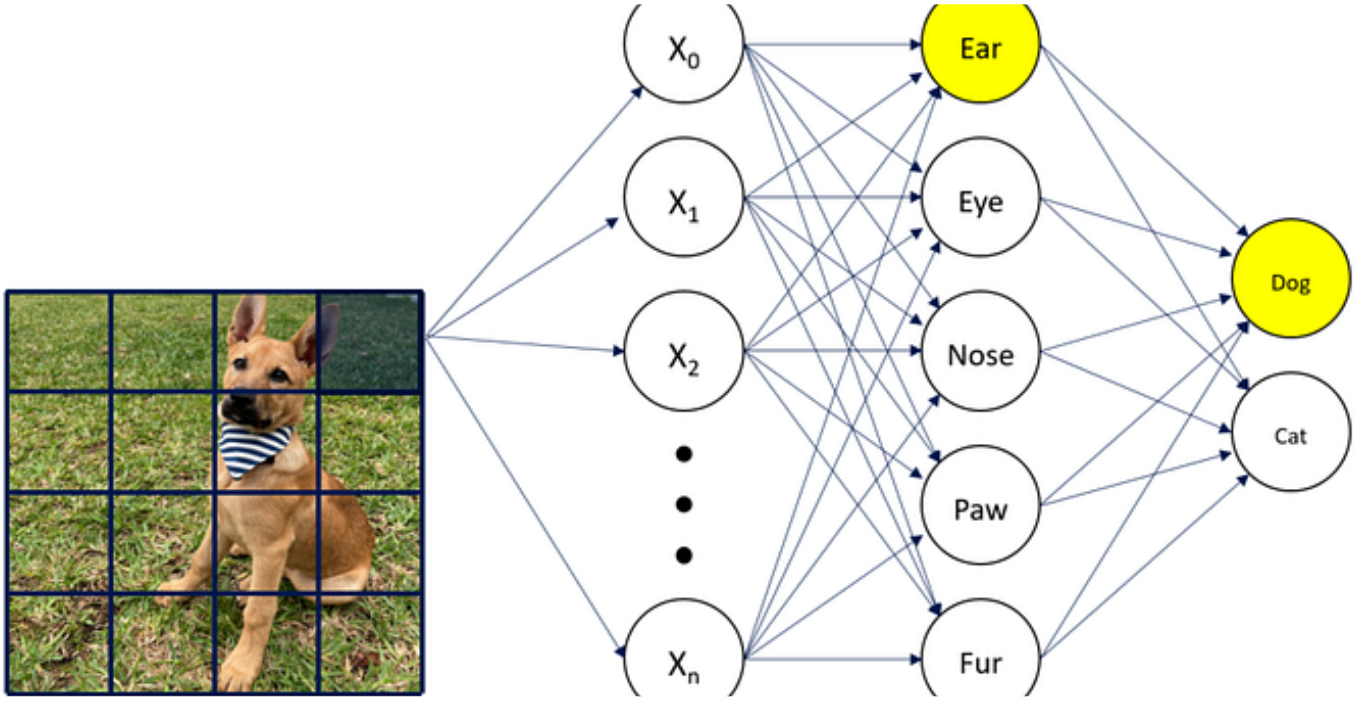
Sudipto Baul in Machine Intelligence and Deep Learning

Transformer: The Self-Attention Mechanism

This post gives a brief overview of the popular self-attention mechanism introduced in 'Attention is All You Need' paper, which is a...

11 min read · May 2, 2022

21



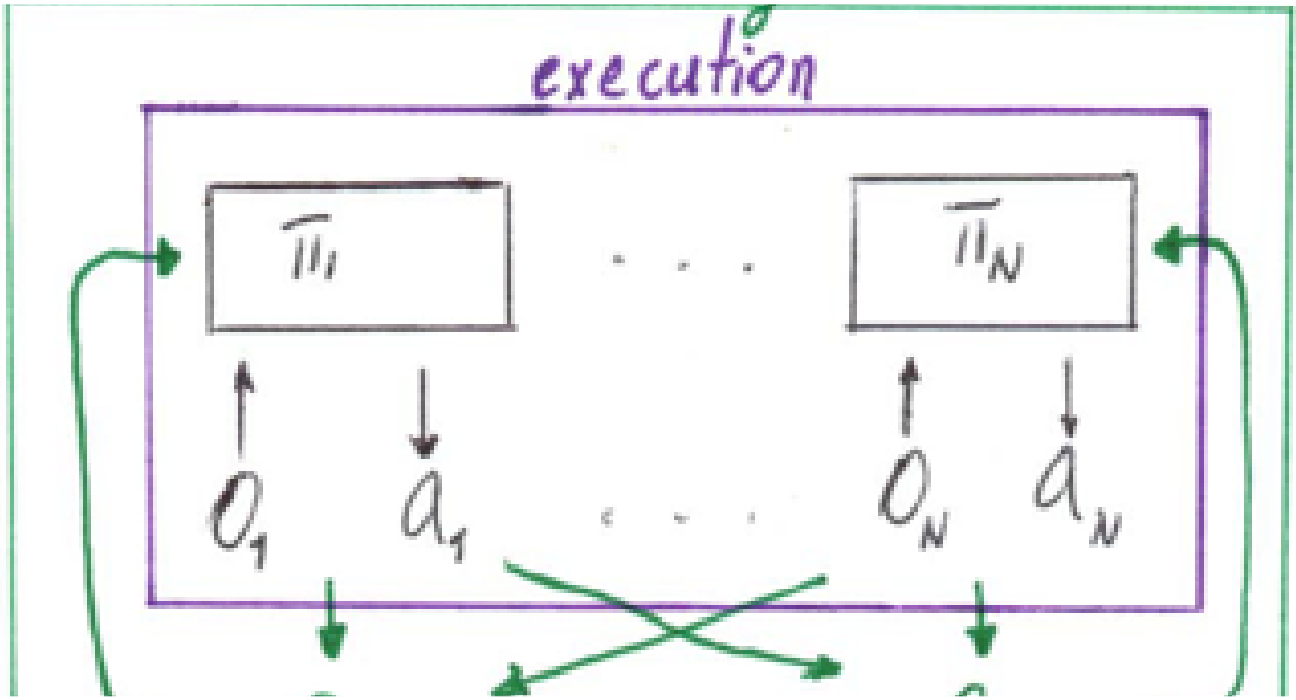
James Laughridge in Machine Intelligence and Deep Learning

Deep InfoMax | Explanation of Mutual Information Maximization

An Introduction of the Deep InfoMax Method for Mutual Information Maximization

8 min read · May 2, 2022

6



Diego Benalcazar in Machine Intelligence and Deep Learning

A tutorial on MADDPG

Original article https://arxiv.org/abs/1911.10635

9 min read · May 2, 2022

2

See all from Azwad Tamir

See all from Machine Intelligence and Deep Learning

Recommended from Medium

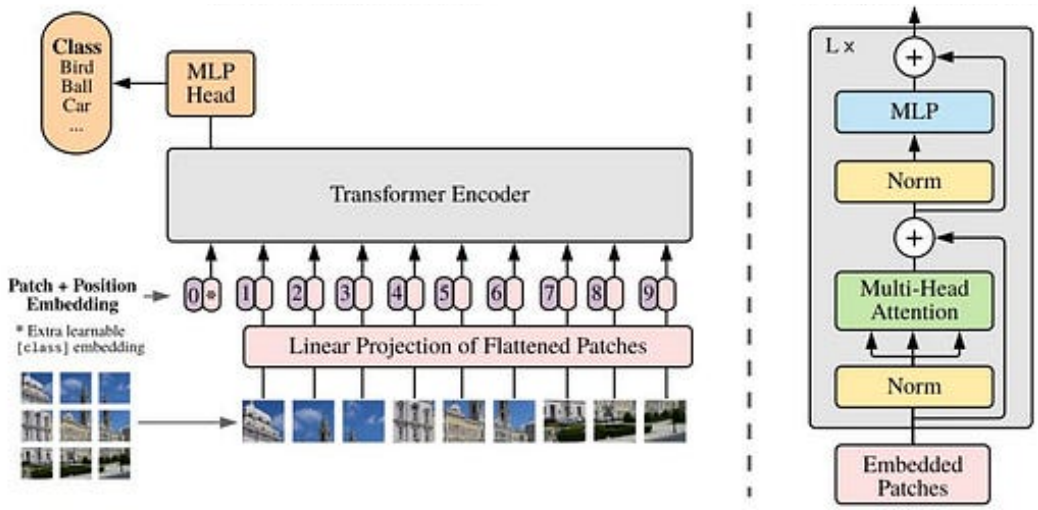


Figure 1: Model overview. We split an image into fixed-size patches, linearly embed each of them, add position embeddings, and feed the resulting sequence of vectors to a standard Transformer encoder. In order to perform classification, we use the standard approach of adding an extra learnable "classification token" to the sequence. The illustration of the Transformer encoder was inspired by

Fahim Rustamy, PhD

Vision Transformers vs. Convolutional Neural Networks

This blog post is inspired by the paper titled AN IMAGE IS WORTH 16×16 WORDS: TRANSFORMERS FOR IMAGE RECOGNITION AT SCALE from google's...

7 min read · Jun 4

385 6

Syed Hamza

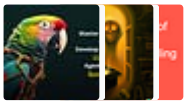
Unlocking the Power of ONNX: A Beginner's Guide with Practical Examples by using 80–20 rule

Discover the game-changing potential of ONNX as we dive into a beginner's guide, packed with practical examples using 80–20 rule.

6 min read · Jul 15

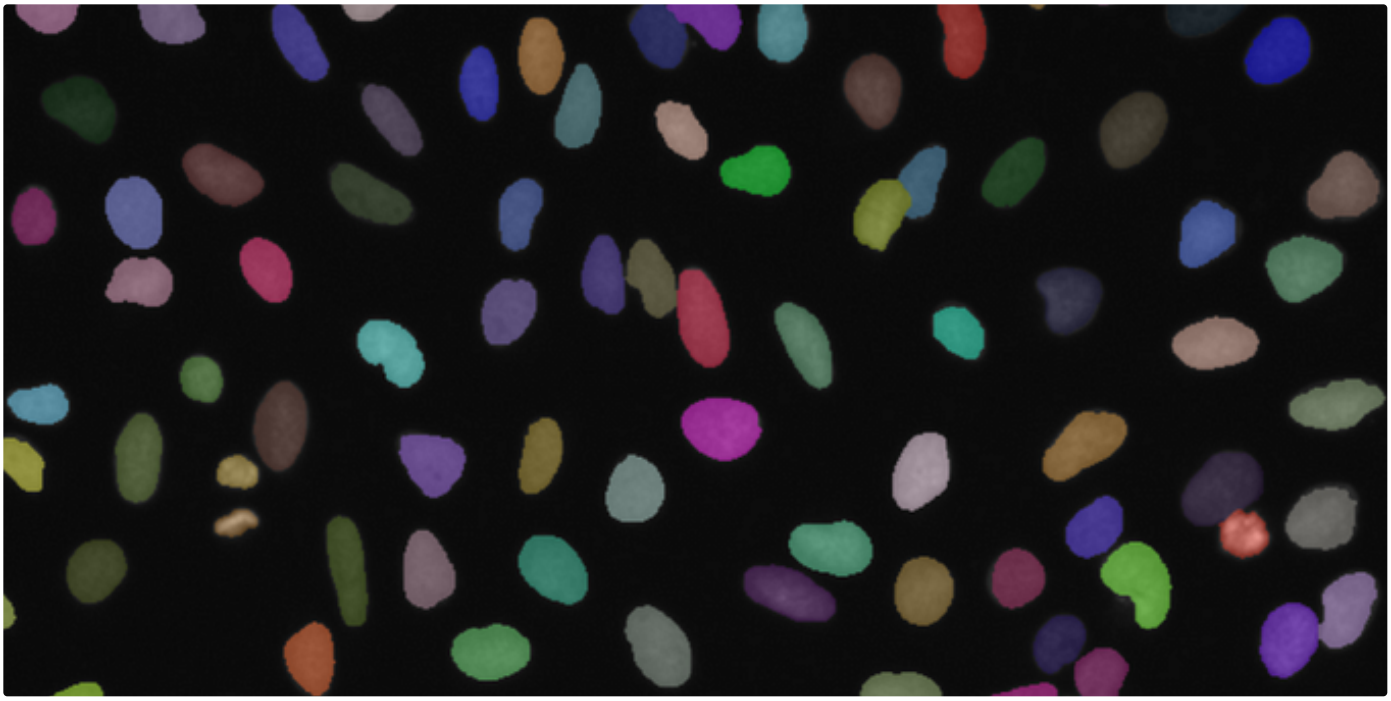
6

Lists



Natural Language Processing

544 stories · 164 saves



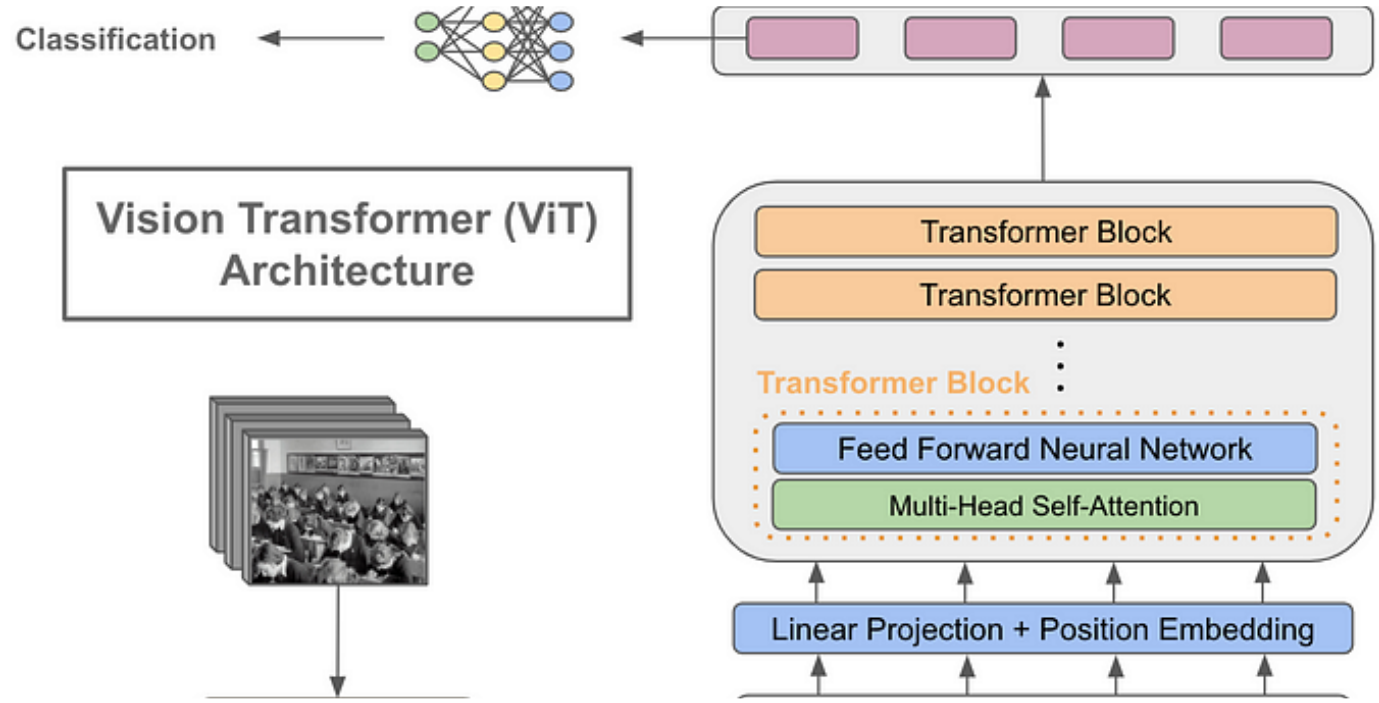
Davy Neven

How to train YOLOv8 segmentation on a custom dataset

A detailed guide on how to use model-assisted labeling to train a YOLOv8 instance segmentation model on the trainYOLO platform.

5 min read · Feb 27

19 1

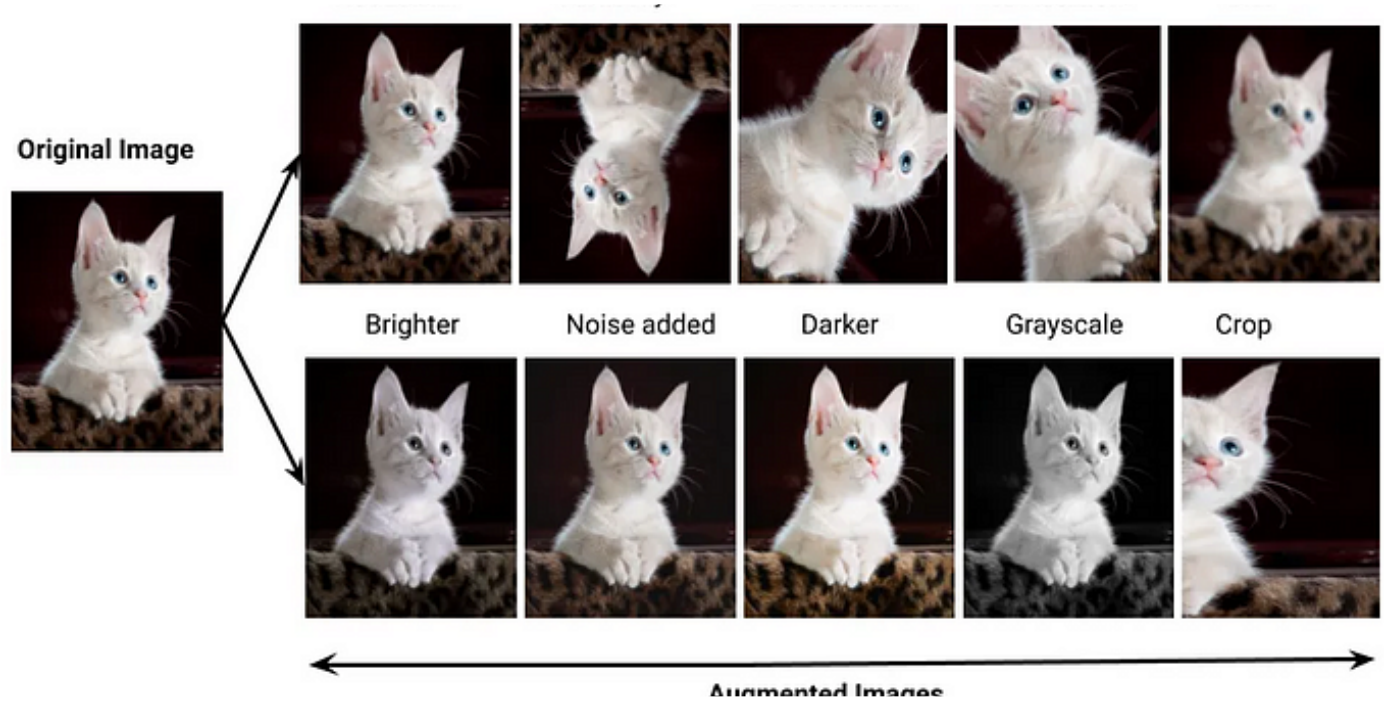


Cameron R. Wolfe, Ph.D. in Towards Data Science

Using Transformers for Computer Vision

Are Vision Transformers actually useful?

· 13 min read · Oct 5, 2022



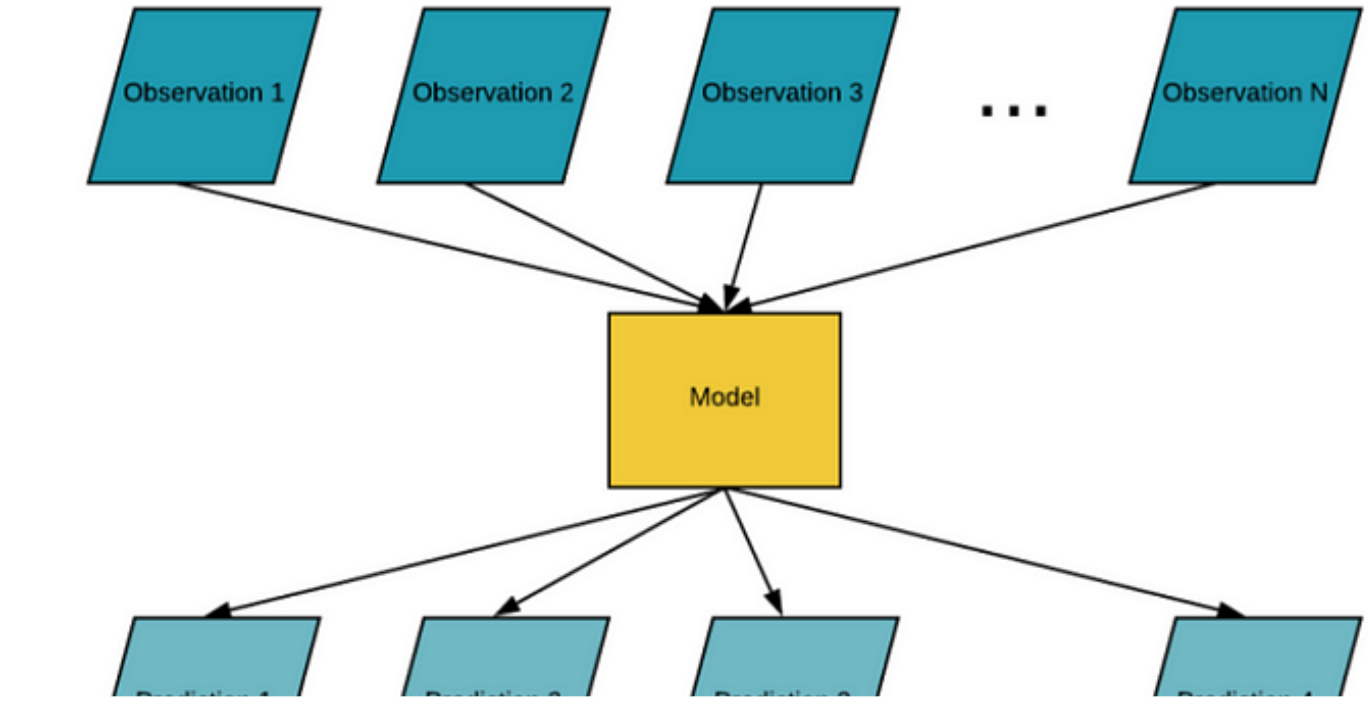
Muhammad Faizan in Red Buffer

Apply Data Augmentation on YOLOv5/YOLOv8 Dataset

How to apply the augmentation on YOLOv5 or YOLOv8 dataset using albumentations library in Python?

3 min read · Jun 6

74



Smaller&Deeper

[yolov8] Batch inference implementation using tensorrt #1— why batch?

Introduction

3 min read · Aug 17

2